

Hybrid AI System for Tender Key Extraction and Validation

1. Project Objective

The objective is to build an AI-powered document processing system that analyzes tender documents and extracts the key fields that will be used by downstream processes.

The system receives tender documents as input:

- PDF
- DOCX
- Word documents

Supported languages:

- French
- English
- Arabic

The system must extract structured business information such as:

- Tender reference
- Submission deadline
- Budget
- Eligibility criteria
- Technical requirements
- Administrative constraints
- Financial conditions
- Required certifications
- Required experience
- Evaluation criteria
- Contact information

The scope of this project stops at:

- Selecting the relevant information
- Extracting the required fields
- Structuring the extracted data
- Validating whether the extracted fields are correct

It does **not** include:

- Deciding whether the tender is suitable or not
- Estimating win probability
- Making business recommendations
- Performing downstream opportunity analysis

The main design principle is:

Do not use an LLM to process the entire document.

Instead, the system uses a hybrid architecture:

- Algorithms handle deterministic extraction
- Transformers handle semantic understanding
- Embeddings handle information selection
- LLMs assist with structured extraction
- A second LLM validates the extracted fields

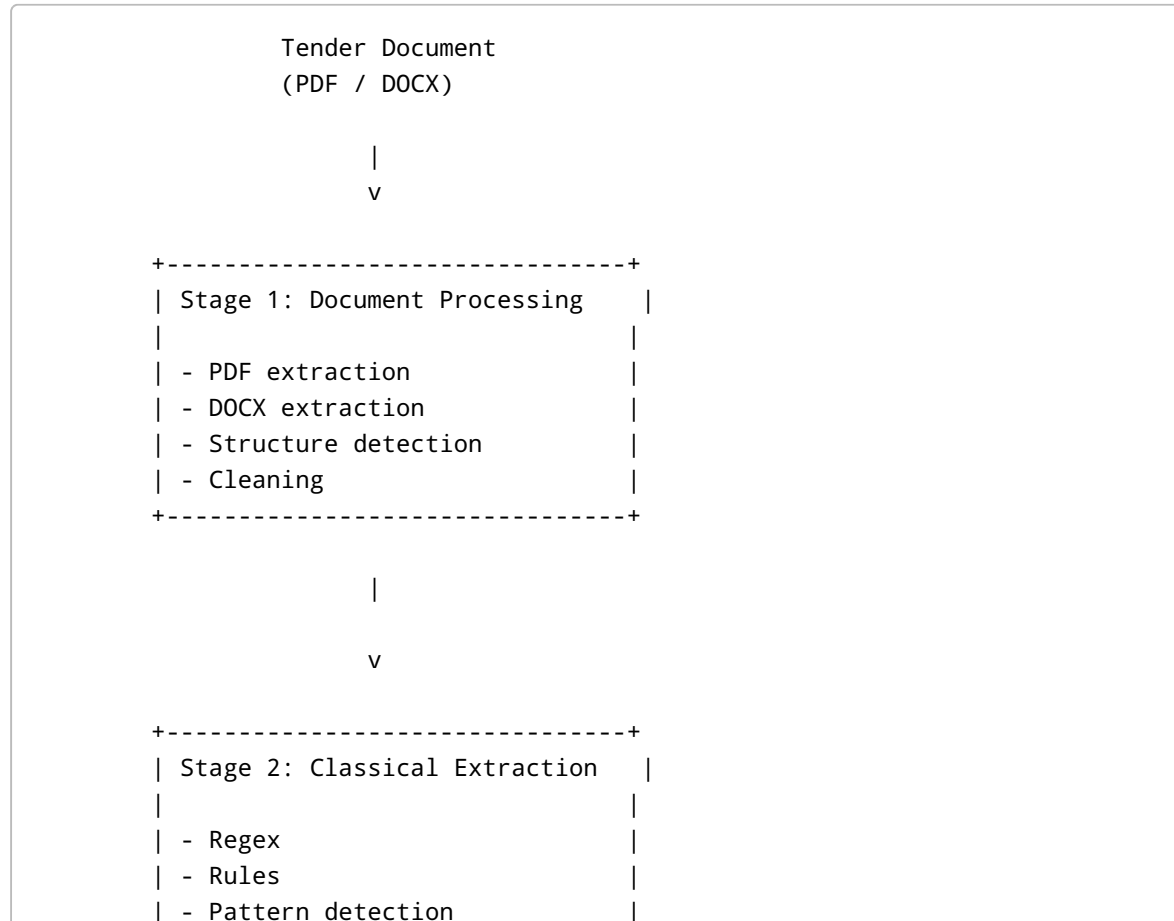
This reduces:

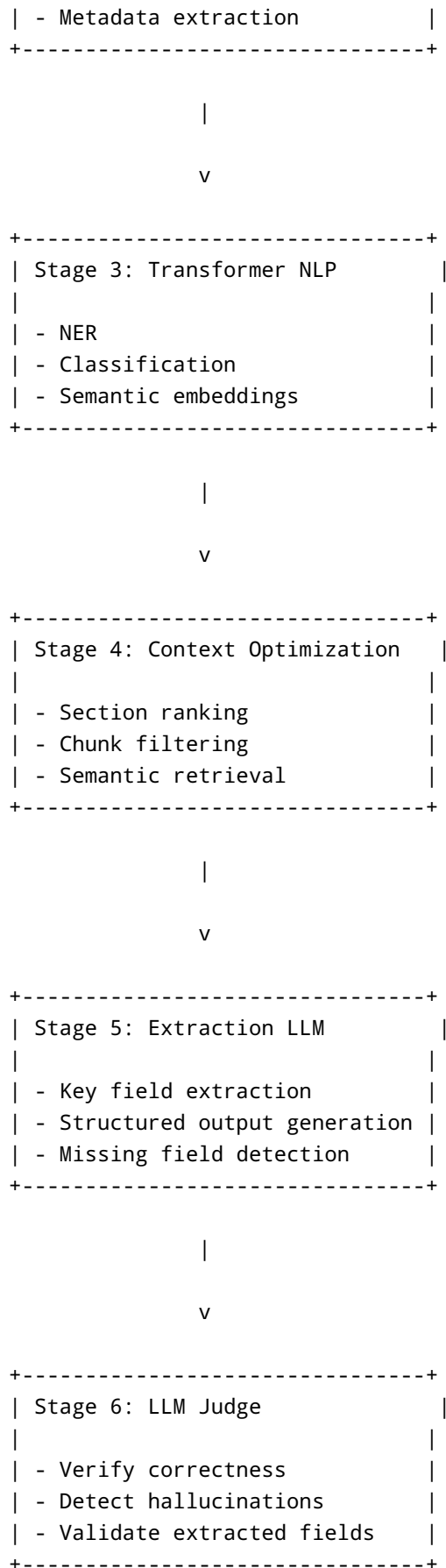
- Token consumption
- API cost
- Processing time

while improving:

- Accuracy
- Explainability
- Reliability

2. System Architecture Overview





|
v
Final Structured Output

3. Stage 1 — Document Processing

Objective

Convert raw documents into structured information.

No OCR is required because documents are digital.

PDF Processing

Recommended tools:

- PyMuPDF
- pdfplumber

Extract:

- Text
- Tables
- Pages
- Sections

Example:

```
{  
  "page":4,  
  "section":"Technical Requirements",  
  "text":"The bidder must..."  
}
```

DOCX Processing

Tool:

- python-docx

Extract:

- Headings
 - Paragraphs
 - Tables
-

4. Stage 2 — Classical Information Extraction

The first extraction layer avoids unnecessary AI usage.

Many tender fields are structured and can be captured with deterministic methods.

Deadline Extraction

Example:

Input:

```
Date limite de dépôt:  
15/07/2026
```

Output:

```
{  
  "deadline": "15/07/2026"  
}
```

Techniques:

- Regex
 - Keyword matching
 - Date parsing
-

Budget Extraction

Detect:

```
500000 DT  
2 million EUR
```

Output:

```
{  
  "budget": "500000 DT"  
}
```

Tender Identification

Detect:

```
AO-2026-001  
Reference: 45892
```

Output:

```
{  
  "tender_reference": "AO-2026-001",  
  "internal_reference": "45892"  
}
```

5. Stage 3 — Transformer-Based Semantic Understanding

Some information cannot be extracted reliably with rules alone.

Example:

Tender requirement:

"The bidder must have completed five similar projects during the last ten years."

Expected:

```
{  
  "type": "experience_requirement",  
  "value": "5 projects",  
  "period": "10 years"  
}
```

Recommended Models

Multilingual NER

Model:

XLNet-RoBERTa

Used for:

- Organizations
- Locations
- Requirements
- Certifications
- Technologies

Embedding Models

Recommended:

- multilingual-e5
- BGE multilingual

Purpose:

Convert text into semantic vectors.

Example:

Company profile or domain profile:

AI software development

Tender section:

Machine learning platform creation

Similarity:

High → keep section

Low → discard

6. Stage 4 — Intelligent Context Compression

Large tenders may contain:

100 pages
50,000 tokens

Sending the entire document to an LLM is inefficient.

The system creates a compressed representation.

Example:

Before:

50,000 tokens

After:

1,500 relevant tokens

Methods

Section Ranking

Prioritize:

- Scope
- Technical requirements
- Eligibility
- Budget
- Deadlines
- Evaluation criteria

Reduce:

- Repeated legal text
 - Boilerplate content
-

Semantic Retrieval

A domain profile or extraction schema is used as a filter.

Example:

Domain profile:

```
AI
Software engineering
Data analytics
```

Tender sections:

```
AI platform development
Similarity: 0.91

Construction project
Similarity: 0.05
```

Only relevant sections continue.

7. Stage 5 — Extraction LLM

The LLM is not used to analyze business suitability.

Its role is limited to helping extract and structure the required fields from the selected context.

It receives:

- Relevant tender sections
- Extraction schema
- Previously detected structured facts

Example input:

```
{
  "tender": {
    "requirements": [
      "5 years experience",
      "ISO certification"
    ],
    "budget": "500000 DT",
    "deadline": "15/07/2026"
  }
}
```

```
}  
}
```

Output:

```
{  
  "deadline": "15/07/2026",  
  "budget": "500000 DT",  
  "requirements": [  
    {  
      "type": "experience_requirement",  
      "value": "5 years"  
    },  
    {  
      "type": "certification_requirement",  
      "value": "ISO certification"  
    }  
  ]  
}
```

The purpose of this stage is:

- Normalize extracted fields
- Fill structured templates
- Capture missing but inferable metadata
- Prepare data for validation

8. Stage 6 — LLM as a Judge

The generated extraction is validated by another model.

The judge does not redo the extraction.

It checks:

- Extraction correctness
- Hallucinations
- Logical consistency
- Missing or unsupported fields
- Whether the extracted value is actually present in the source

Example:

Extractor:

Company has ISO certification

Judge:

No evidence found in the source document

Result:

```
{  
  "issue": "Unsupported claim",  
  "severity": "high"  
}
```

The judge can also verify formatting and normalization.

Example:

Extractor:

```
{  
  "deadline": "July 15, 2026"  
}
```

Judge:

```
{  
  "issue": "Date format not normalized according to schema",  
  "severity": "medium"  
}
```

9. Multi-Model Strategy

Avoid using the same model for both roles.

Recommended:

Extraction LLM

Local:

Qwen2.5

Versions:

- qwen2.5:7b
- qwen2.5:14b

Advantages:

- Multilingual
 - Strong structured extraction
 - Good Arabic support
-

Judge

Different model:

Examples:

- Mistral
- Llama

Reason:

Different models reduce shared mistakes and make validation more reliable.

10. Self-Correction Loop

For uncertain cases:

```
Extractor
|
v
Judge
|
|
Problems found
|
v
Extractor revision
|
v
Final structured output
```

11. Local Deployment Architecture

Backend

Python

- FastAPI

Document Processing

- PyMuPDF
- pdfplumber
- python-docx

NLP

- Transformers
- spaCy
- sentence-transformers

Vector Database

Local:

- FAISS
- ChromaDB

LLM Runtime

Local:

- Ollama
- Qwen models

Alternative:

Groq API for faster inference.

12. Token Optimization

Example:

Large tender:

Before:

50,000 tokens

After extraction:

500 tokens

After retrieval:

1000 tokens

LLM receives:

1500 tokens

Reduction:

Approximately 95%

13. Confidence-Based Routing

Not every tender needs the same level of validation.

Simple cases:

High confidence
|
v
Direct structured output

Complex cases:

Low confidence
|
v
LLM Judge

Examples requiring validation:

- Mixed languages
- Contradictory requirements
- Ambiguous clauses

- Poorly structured documents
 - Conflicting dates or references
-

14. Development Roadmap

Phase 1 — MVP

Build:

```
Document extraction
  |
Cleaning
  |
Regex extraction
  |
Embeddings
  |
Qwen extraction
```

Goal:

Validate the extraction pipeline and structured output quality.

Phase 2 — Advanced NLP

Add:

- XLM-R NER
 - Requirement classification
 - Better semantic extraction
-

Phase 3 — Extraction Intelligence

Build a field extraction knowledge base:

- Common tender fields
- Certification patterns
- Deadline formats
- Administrative clauses
- Technical requirement templates

Improve:

- Field coverage

- Extraction precision
 - Validation quality
-

Final Architecture Principle

The system follows:

Algorithms extract facts

Transformers understand meaning

Embeddings select useful information

LLM extracts structured fields

LLM Judge validates reliability

The LLM is used as a controlled extraction and validation component, not as a document-wide reasoning engine and not for business suitability decisions.